

ArgorixLang: Compiled Governance, Sovereign Agent Metadata, and Offline-Verifiable Runtime Evidence

Gustavo Venegas¹ Edison Vazquez¹ Danilo Naranjo² Benjamin Gonzalez¹

¹Chilean Chamber of Artificial Intelligence ²Ocular

June 2026

Abstract

Agent systems increasingly combine model calls, tools, data movement, and inter-agent protocols, but their governance assumptions are commonly expressed outside the executable artifact. This paper presents ArgorixLang, a compiled language and runtime in which agent capabilities, policy, sovereign metadata, provider boundaries, and evidence requirements are explicit program constructs. We distinguish four claim classes throughout: implemented mechanisms, declarative metadata, proposed architecture, and properties not claimed. We study an authorized corpus of 33 runtime request directories using a reproducible, observational pipeline. Twenty-seven directories contain all five expected artifacts and six contain source only. All 27 complete evidence bundles pass offline digest and cross-artifact verification of bytecode, trace, and security report, including a ledger digest recomputed from trace events and matched against the report’s LedgerSummary; source integrity is outside bundle verification. At the same time, all 27 detailed policy reports set `passed=false`, require review, and record 44 unknown-rule violations each, despite a favorable top-level `security_checks` label. Thus bundle integrity does not imply policy approval, security, or certification. The corpus contains 7,155 ledger events and 1,188 detailed policy violations. The bounded `injected.content` field is present in 0 of 27—that is, none of the 27—complete, valid traces; six source-only directories have no trace and are not assessable, precluding prompt-level qualitative analysis. We describe the compiler, fail-closed runtime, Agent Passport metadata, a declarative encoding of the unpublished ATrust conceptual protocol, its relationship to DCP-AI, and trace/report/bundle generation. The results demonstrate reproducible artifact production and offline integrity checking under the observed configuration; they do not demonstrate real identity authentication, credential verification, cryptographic handshakes, blockchain immutability, post-quantum security, compliance certification, or production isolation.

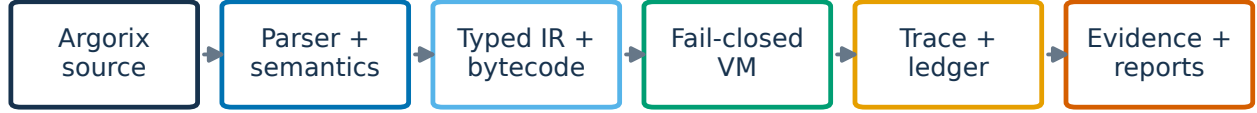
Keywords: agent programming languages; compiled governance; sovereign identity; runtime evidence; fail-closed execution; reproducibility

1 Introduction

An agentic application is not merely a model invocation. It is a composition of principals, messages, provider operations, tools, policy decisions, and evidence. When these concerns are distributed among prompts, deployment configuration, and application code, a reviewer cannot determine from one artifact which operation was authorized, which boundary was enforced, or which claim is only aspirational. Protocols such as MCP and A2A standardize useful interaction surfaces [1, 6], while identity and credential standards define interoperable data models [17, 18]. They do not by themselves make an application policy executable or its local execution evidence reproducible.

Existing work demonstrates complementary separations: OPA decouples policy decision from enforcement [8], Jason provides an agent-oriented language and interpreter [3], in-toto verifies software-supply-chain provenance [14], and WASI defines capability-oriented host interfaces [16]. Our question is narrower

Argorix compilation and evidence architecture



Offline runtime boundary • simulated is the sole executable provider

Figure 1: The governed execution and sovereign trust stack. Solid paths denote implemented compilation and runtime behavior; metadata layers do not imply external verification.

and compositional: what can be compiled and evidenced together in one governed agent runtime without upgrading metadata into external assurance?

ArgorixLang addresses this systems boundary by compiling declarations for agents, messages, passports, policies, providers, governance, and evidence into a validated intermediate representation and bytecode executed by a constrained virtual machine [2]. The core design decision is to retain source-faithful declarations until semantic validation, then lower only validated constructs. Runtime decisions produce a trace, a SecurityReport, and an EvidenceBundle containing semantic SHA-256 digests for bytecode, trace, the security report, and the trace event ledger. Source remains an upstream input outside bundle verification. An executable provider registry is separate from declarative external-provider contracts. In the studied configuration, only `simulated` is executable.

The paper asks five explicit research questions:

RQ1. How can governance, sovereign identity, and evidence requirements be represented as compiled language constructs?

RQ2. How does the ArgorixLang pipeline constrain a chatbot request before a provider operation can be planned?

RQ3. What evidence does the runtime generate, and how can its internal consistency be checked offline?

RQ4. What do the existing runtime sessions demonstrate, and what do they not demonstrate?

RQ5. How can Agent Passports, ATrust, DCP-AI, and future sovereign discovery fit into an open agentic-web architecture?

Our contributions are: (1) a language architecture that exposes governance and sovereign metadata to compilation; (2) a fail-closed runtime/provider boundary with explicit negative controls; (3) an evidence model separating artifact integrity from policy judgment; (4) a reproducible observational study that reports complete, incomplete, and internally inconsistent outcomes; and (5) a claim-bounded roadmap connecting implemented passport identifiers to proposed sovereign discovery.

Figure 1 summarizes the stack. The empirical result is deliberately narrower than an assurance claim:

we observe deterministic structure, bounded execution, and verifiable artifact relationships in one corpus. We neither conduct a controlled attack experiment nor infer production security from successful digest verification.

Where a figure and table address the same topic, they are deliberately paired rather than duplicative: the figure communicates topology, flow, or boundary placement, while the table provides exact categorical or quantitative comparison. Captions and prose identify the distinct evidentiary role of each.

2 Background and Related Work

Open agent discovery. Project NANDA’s current official title is *Architecting the Internet of AI Agents* [10]. Its research anchor, “Beyond DNS: Unlocking the Internet of AI Agents via the NANDA Index and Verified AgentFacts,” proposes indexing and verifiable facts for networked agents [11]. This work motivates an open discovery layer, but ArgorixLang does not implement NANDA, DNS resolution, or a remote index. The comparison is architectural: locally compiled sovereign metadata could become an input to a future discovery service.

Names, identifiers, and credentials. DNS separates hierarchical naming from the mechanisms that applications build above it [4, 5]. DID Core specifies identifiers and DID documents, while the Verifiable Credentials Data Model specifies a credential data model [17, 18]. Declaring a string shaped like a DID or a credential reference is not equivalent to resolving, authenticating, issuing, or verifying it. We use that distinction throughout.

Agent interaction protocols. MCP defines a protocol for context and tool interactions; A2A defines inter-agent communication [1, 6]. ArgorixLang models bindings and bridge constraints for these boundaries declaratively. The observed runtime records that bridge runtime, network, tool, or agent execution is disabled. It therefore tests declarations and negative controls rather than interoperability with live MCP or A2A endpoints.

Risk framing. NIST AI RMF organizes AI risk management as a lifecycle activity [13], and OWASP catalogs application risks including prompt injection and sensitive-information disclosure [9]. These sources inform threat categories, not certification mappings. A policy name or governance record cannot establish legal compliance.

Policy as code and agent languages. Open Policy Agent (OPA) separates a general-purpose policy decision engine from application enforcement and expresses decisions in Rego [8]. ArgorixLang instead compiles policy together with agent, passport, provider, and evidence declarations and emits VM-local decision events; it is not an OPA implementation or Rego evaluator. Jason interprets an extension of AgentSpeak for belief–desire–intention multi-agent programming [3]. Jason’s primary concern is agent reasoning and multi-agent execution, whereas this study examines compiled governance, provider denial, and runtime evidence. ArgorixLang does not claim Jason compatibility or equivalent BDI semantics.

Provenance and isolation. in-toto cryptographically records and verifies software supply-chain steps [14]. ArgorixLangEvidenceBundles concern one local runtime result, use unsigned semantic digests, and exclude source; they neither implement in-toto layouts nor provide its signer-based supply-chain guarantees. WASI specifies standards-track host APIs and a capability-based sandbox in which authority is explicitly granted [16]. The current ArgorixLang profile denies operations at the VM/application layer but does not run inside, implement, or empirically compare a WASI sandbox.

Table 1: Scope of related-work comparisons. Each row is relational, not an equivalence or conformance claim.

System	Comparison dimension	Bibliographic scope
Project NANDA	Agent naming and discovery	Project-level comparison only
ATrust	Agent trust relationships	Project-level comparison only
DCP-AI	Discovery and control plane	Project-level comparison only
Open Policy Agent	External policy-as-code engine	Direct architectural comparison
Jason / AgentSpeak	BDI agent language and interpreter	Direct language/runtime comparison
in-toto	Signed supply-chain provenance	Direct evidence-model comparison
WASI	Capability-based host isolation	Direct isolation-boundary comparison

Table 2: Representative construct families and their boundary in the studied implementation.

Construct family	Representation	Boundary
Agents and messages	Typed declarations	Implemented
Provider contracts	Declarative metadata	Declarative
Policies and passports	Semantic plus runtime controls	Implemented
ATrust evidence maps	Validated evidence links	Declarative

Table 1 pairs each system with one comparison dimension. Its rows are exact side-by-side comparisons, while the surrounding figure program shows architectural topology; neither claims implementation equivalence, protocol conformance, or shared assurance.

The language is implemented in Rust [12]. The evolution in figure 2 reflects a sequence from agent execution toward typed messages, provider contracts, passports, ATrust metadata, governance, hardening, and evidence. Version numbers establish repository history only; they are not external certifications.

3 Language and Compiler Architecture

ArgorixLangprograms group declarations into modules. Construct families include agents and capabilities, typed messages and handlers, tools and model calls, policies, provider contracts, passports, identity and credential metadata, handshakes, trust ledgers, evidence maps, governance profiles, threat models, runtime profiles, and release/conformance records. Some constructs alter runtime behavior; others are validated descriptions whose execution is explicitly disabled.

Table 2 shows why “represented in the language” is not one claim: typed agents and runtime policy have implemented behavior, whereas ATrust evidence maps and provider contracts remain validated descriptions at the boundary discussed here.

The pipeline has five relevant stages. Parsing produces a source-faithful abstract syntax tree. Name and module resolution bind declarations without discarding provenance. Semantic analysis checks uniqueness, references, cross-construct constraints, and denied combinations. Lowering converts only validated forms to intermediate representation and serializable bytecode. Finally, the VM loads bytecode, installs explicitly executable providers, and evaluates policy before scheduling work.

This order matters. A runtime check alone discovers invalid configuration after deployment; an unchecked lowering pass can erase the distinction between an omitted field and an approved field. Semantic validation rejects, among other cases, unknown references, duplicated names, external execution

Language evolution and bounded future work

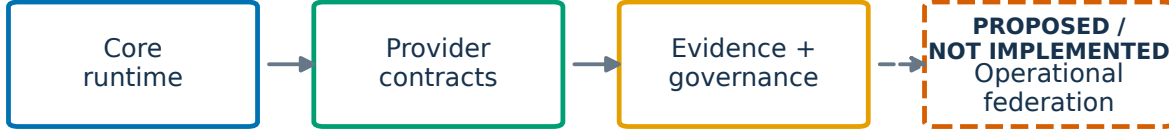


Figure 2: ArgorixLang language evolution represented by repository versions and feature boundaries. The timeline establishes implementation sequence, not maturity or deployment readiness.

without required approval, inconsistent provider/capability bindings, permissive runtime profiles, and trust declarations that assert prohibited security claims. The conformance corpus contains positive and negative examples for these boundaries.

Figure 3 answers RQ2 at the architectural level. A request cannot directly select an arbitrary provider. The VM resolves the agent and handler, evaluates policy, checks capabilities, and consults the executable provider registry. Declarative contracts such as `OpenAIProvider` remain descriptions unless an independently executable adapter is registered. In the observed configuration, the registry contains `simulated`; attempts to cross the external boundary are recorded as blocked.

The language also preserves negative statements. Fields such as `network_allowed=false`, denied key material, mandatory audit, and fail-closed policy are not comments; their combinations are checked. However, the presence of these fields proves only that the compiler accepted the declaration and the runtime recorded its configured handling. It does not prove operating-system containment.

4 Sovereign Identity Metadata

An Agent Passport collects locally asserted identity and governance context: passport name, subject agent, country, jurisdiction, permitted data residency, capabilities, policy bindings, and an `ans_name`. Semantic validation ensures that references resolve and that enumerated relationships are well-formed. Runtime traces can therefore identify which passport and residency constraints accompanied an operation.

The term *sovereign* refers here to explicit jurisdictional and residency metadata controlled by the program, not state recognition or legal status. The observed complete sessions each contain two passports, jurisdiction CL, country CL, and residency values CL and EU. These values are declarations. No external registry validates them.

`ans_name` is implemented as a declarative passport identifier. It survives parsing, semantic validation, lowering, and artifact generation. It is not an operational Agent Naming Service: no DNS query, DID

Request lifecycle: controls precede execution

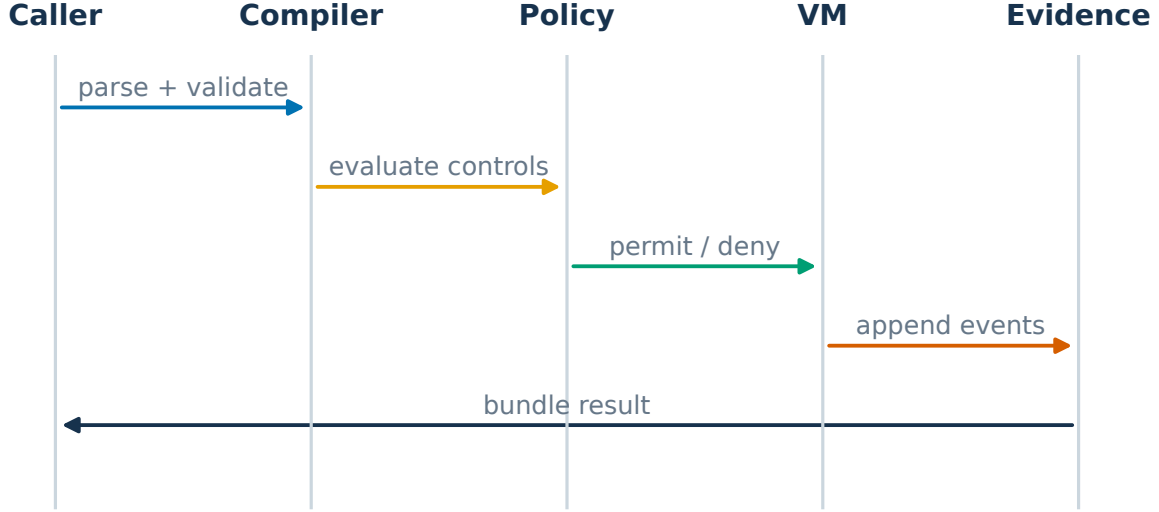


Figure 3: End-to-end request sequence. Semantic validation and policy evaluation precede operation planning; evidence is produced from the resulting runtime state.

resolution, registry lookup, ownership challenge, or remote authentication occurs. Figure 4 makes the implemented relation explicit: the name is bound to a local passport and agent within the compiled program.

This separation avoids a common category error. A resolvable name answers “where is the record?”; an authenticated identity answers “who controls the relevant key or account?”; a credential answers a scoped assertion; and a policy decision answers whether an operation is permitted in context. A future architecture may connect these functions, but one cannot substitute for another.

Figure 5 places the operational cut immediately after local compilation: federation, DID resolution, and remote proof of control are future components and cannot be inferred from a validated `ans_name`.

5 ATrust and DCP-AI

ATrust is attributed to Edison Vazquez as an unpublished conceptual protocol [15]. In the conceptual model, an agent identity relates to credential descriptions, handshake requirements, trust-ledger entries, and an evidence map. ArgorixLang implements syntax, semantic relationships, bytecode representations, and trace events for these concepts. It does not implement real credential issuance or verification, cryptographic challenge–response, remote DID resolution, or live attestation.

Identity declarations associate agents with identifier methods and boundaries. Credential contracts describe issuer/subject/type relationships. Handshake declarations describe participants and required evidence. A trust ledger is a local, hash-linked declarative structure whose entries reference subjects and evidence. Evidence maps require named declarations to be covered by artifact links. Validation rejects dangling references and prohibited claims. The word *handshake* therefore denotes a checked contract, not an executed cryptographic exchange; *ledger* denotes a local evidence data structure, not blockchain consensus or immutable storage.

DCP-AI, attributed to Danilo Naranjo Emparanza, proposes a Digital Citizenship Protocol for Au-

Declarative trust relationships



Figure 4: Relationships among passports, agents, policies, ATrust declarations, and evidence. Edges indicate validated references, not verified real-world relationships.

tonomous AI and a control-plane framing for agents [7]. It complements ATrust’s trust relationships and ArgorixLang’s compiled enforcement boundary, but the three are not conflated. DCP-AI is a published conceptual proposal; ATrust is unpublished conceptual work; ArgorixLang is the software artifact evaluated here.

The relationship can be stated operationally. DCP-AI motivates questions of agent standing and governance; ATrust supplies a vocabulary for identity, credentials, handshakes, ledgers, and evidence; ArgorixLang checks local declarations and emits evidence about their handling. None of these local checks demonstrates that a remote party is genuine.

Figure 6 therefore distinguishes the dashed, unverified source input from the bundle-verified JSON artifacts. Its scope does not extend to an issuer, credential authority, or remote agent.

MCP and A2A bridges follow the same principle. They bind agents, passports, identities, message contracts, and boundaries while requiring network and execution denial in the studied profile. The declarations can support future protocol adapters [1, 6]; today they are declarative boundaries and auditable negative controls.

6 Governed, Fail-Closed Runtime

The runtime evaluates a validated program rather than accepting unrestricted provider calls. Its decision inputs include the requesting agent, capability, target, policy, runtime profile, and executable-provider registry. Denial is the default when a reference is unknown, an approval is absent, a capability is incompatible, or a provider is not executable. Runtime state is retained in an outcome object even when execution fails, allowing a report to describe the denial without converting it into success.

The provider boundary has two registries. Declarative contracts state capabilities, limits, and allowed operations for an external provider. Executable providers contain callable implementations. A contract’s presence does not enroll it for execution. The evaluated chatbot artifacts name an `OpenAIProvider`

Sovereign discovery boundary

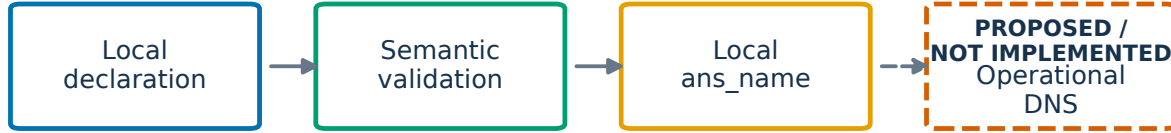


Figure 5: Implemented and proposed discovery boundary. Local `ans_name`, passport, and policy bindings are implemented; Sovereign DNS/ANS federation, DID resolution, and remote discovery are proposed.

Table 3: Runtime controls observed in the evaluated profile. These controls describe application-level behavior, not operating-system sandbox proof.

Control	Observed behavior	Scope
Provider execution	simulated only	Implemented
External execution	Blocked	Implemented
Network access	Denied by runtime profiles	Implemented
Secrets and key material	Denied	Implemented

contract and an `OpenAISandbox` descriptor, but the only executable provider is `simulated`. Each complete trace records one blocked external-provider attempt.

Table 3 records the four controls actually observed, while leaving operating-system containment and live-provider behavior outside the table’s scope.

Runtime profiles require audit, evidence, security reports, sandbox metadata, and deny network, plaintext secrets, key material, tools, agent execution, and external execution for the demonstrated path. Governance and threat-model declarations are likewise runtime-disabled. These choices make the corpus suitable for studying compilation and evidence without contacting an external service.

Figure 7 also clarifies the meaning of *fail closed*. It is a control-flow property of the implemented VM: missing authorization does not fall through to a provider call. It is not a claim that every host-language, dependency, operating-system, or deployment path is isolated. A production assessment would need process, filesystem, network, secret-store, and adapter testing beyond the present study.

Figure 8 makes unavailable source-only observations visually distinct from favorable results; this prevents missing artifacts from being counted as control passes. Policies can activate actions, emit violations, and require human review. Coarse status fields are not authoritative when a detailed policy object is available. This rule becomes consequential in section 9: the corpus contains a favorable top-level

Offline evidence verification scope



Bundle checks bytecode, trace, and report digests

`ledger_digest = digest(trace.events); report LedgerSummary must match`

Figure 6: Offline evidence verification scope. Source is upstream and outside bundle verification; verified inputs are bytecode, trace, and security report, with the ledger digest derived from trace events and matched to the report’s LedgerSummary.

label alongside uniformly unfavorable detailed policy results.

7 Trace, Report, and Evidence Model

Each complete request directory contains five artifacts: `session.argx`, compiled `session.argbc.json`, `session.trace.json`, `session.security.json`, and `session.evidence.json`. Source is an upstream compiler input, not an EvidenceBundle member. The trace is an ordered ledger of compiler/VM and policy events. The SecurityReport is a structured interpretation of runtime evidence, including execution status, policy results, review state, provider boundary, and declared controls. The EvidenceBundle records four semantic SHA-256 values: `bytecode_digest`, `trace_digest`, `report_digest` for the security report, and `ledger_digest`, which is the canonical digest of `trace.events`. Its artifact map contains `bytecode_path`, `trace_path`, and `security_report_path`; there is no source path or source digest, and there is no standalone ledger path.

A semantic digest is computed over canonical JSON derived from a deserialized value, avoiding dependence on whitespace or object-key formatting. The three artifact paths resolve relative to the bundle directory. Offline verification loads bytecode, trace, and the security report and recomputes their three semantic digests. It separately computes `ledger_digest` from `trace.events`. Finally, it checks that the `ledger_digest` stored in the SecurityReport’s LedgerSummary equals the bundle’s `ledger_digest`; the report stores this summary digest, not ledger events. It requires neither network access nor provider credentials. This verifies the relationships encoded by the bundle. Source integrity is outside EvidenceBundle verification.

Figure 9 exposes this asymmetry: a complete request directory contains source, but the bundle verifier starts at compiled bytecode and never asserts that `session.argx` produced it.

Three distinctions are essential. First, a valid digest detects content change relative to the bundle but

Table 4: Interpretive boundaries applied to every result in this study.

Concept	Implemented	Declarative	Proposed	Not claimed
Provider boundary	simulated allowlist	contract metadata	–	external execution
Trust evidence	digest bundle	ATrust map	–	live attestation
Discovery	local ans_name binding/metadata	sovereign meta- data	federation	operational DNS
Security scope	fail-closed controls	threat mappings	deployment study	certification

does not prove who created the bundle. Second, a linked trace does not prove that events outside the instrumented runtime did not occur. Third, artifact integrity does not turn a failing policy result into approval. No signature, trusted timestamp, transparency service, hardware root, or independent witness is evaluated here.

The local trust ledger is similarly bounded. Hash-linked entries and validated references can expose accidental or adversarial modification when the verifier has an expected root or bundle. They do not provide distributed consensus, blockchain immutability, or non-repudiation. The declarations explicitly deny such security claims.

Figure 10 is the interpretive key for the remainder of the paper: each result is assigned to an implemented, declarative, proposed, or not-claimed region before it is discussed.

Table 4 applies the same distinction to concrete system concepts, preventing digest success from being promoted to live attestation or certification.

8 Methodology

8.1 Study design

We conduct an observational, reproducible, non-controlled systems study. The unit of analysis is one request directory in an authorized snapshot of the chatbot runtime’s generated output. The user authorized publication of the runtime material. Publication authorization does not relax minimization: identifiers are aggregated, excerpts are bounded, and prompt content is not reconstructed.

The design is observational because sessions were generated before the analysis rather than assigned to controlled treatments. It is reproducible because committed scripts normalize the snapshot, generate tables and figures, and rerun offline bundle verification. It is non-controlled because there is no randomized input, baseline runtime, repeated attack suite, or experimental manipulation. Consequently, counts describe this corpus and cannot estimate attack success, causal effects, or population rates.

8.2 Inventory and normalization

The initial inventory contains 33 directories. A complete directory must have all five expected artifacts listed in section 7; 27 meet that criterion and six contain only `session.argx`. We retain incomplete directories in the denominator and represent unavailable values as missing, rather than silently discarding them.

Table 5 separates session-level normalization, event-level counts, and verifier outcomes so that totals can be recomputed without treating one row type as another. The analysis parses available JSON, records parse errors, counts sizes and artifact presence, extracts execution and policy fields, counts ledger-event kinds, and records passport, runtime-profile, adapter, provider-boundary, and digest metadata. It writes one session row to `sessions.csv`, one session/event-kind row to `event_counts.csv`, and a structured summary to `runtime_summary.json`. A separate PowerShell procedure invokes the repository’s offline evidence verifier for every complete bundle and stores exit code and bounded standard output.

Table 5: Committed normalized datasets. Values are regenerated from the authorized artifact snapshot.

Normalized source	Rows / records	Role
runtime_summary.json	33	Session-level normalized summary
sessions.csv	33	Tabular session observations
event_counts.csv	1890	Per-session event-kind counts
verification-results.json	27	Verification result groups

8.3 Analysis rules

We define in advance four interpretation rules. A missing artifact is “unavailable,” never “passed.” A detailed policy result overrides a coarser summary label for policy interpretation. Successful evidence verification means digest and reference consistency only. Declarative records are not counted as externally performed actions.

We aggregate complete-session policy violations and ledger events by summing the normalized per-session fields. We check all four evidence digest values for the `sha256:` prefix and 64 hexadecimal digits. The verification result is successful only when the verifier exit code is zero and its explicit verified flag is true.

8.4 Prompt analysis boundary

Although the user authorized publication, we inspect only the bounded `trace.injected.content` schema field. It is absent from all 27 valid complete traces (0 of 27 present). The six source-only directories contain no trace and are not assessable. Prompt publication is a separate allowlisted operation; its suppression is not evidence of content absence. We therefore perform no prompt-level qualitative analysis, quote no prompts, and report no prompt-injection attack success rate. Inferring prompts from request identifiers, trace shape, or response metadata would be methodologically unsound and privacy-increasing.

8.5 Threats to validity

Construct validity is limited by our mapping of runtime fields to concepts such as “policy result” and “blocked attempt.” We mitigate this by preserving field names and reporting contradictions. Internal validity is weak by design: no causal intervention exists. External validity is limited to one snapshot and one runtime configuration. Reliability depends on the committed normalizer and verifier; generated CSV/JSON files permit independent checking. Researcher interpretation remains a risk, particularly when declarative and operational concepts share terminology.

9 Results

9.1 Completeness and execution

Of 33 request directories, 27 (81.8%) contain all expected artifacts and six (18.2%) contain source only. All 27 complete sessions report execution status `completed`; execution status is unavailable for the six incomplete sessions. The distinction matters: the study cannot determine whether those six requests failed during compilation, were interrupted, or were retained before downstream artifacts were written.

Figure 11 keeps the six source-only directories in view, while table 6 reports aggregate totals only where normalized values exist; together they prevent complete-case counts from becoming a 33-session success claim.

Table 6: Empirical totals derived from the normalized datasets.

Measure	Derived value	Source
Complete sessions	27	runtime_summary.json
Incomplete sessions	6	runtime_summary.json
Normalized sessions	33	sessions.csv
Complete-session policy violations	1188	sessions.csv
Counted ledger events	7155	event_counts.csv
Prompt-content traces inspected / present	27 / 0	runtime_summary.json
Verified evidence bundles	27 / 27	verification-results.json

9.2 Policy inconsistency

Each of the 27 complete sessions has `security_checks=passed` at the top level. Nevertheless, each detailed policy object has `policy_passed=false`, `review_required=true`, and 44 violations, all carrying the reason “unknown policy rule.” The total is therefore 1,188 policy violations. These rules cover ATrust, trust-ledger, MCP, and A2A checks, among others.

We treat this as an inconsistent outcome, not a successful policy evaluation. The top-level field may describe a narrower check or stale aggregation, but the artifacts do not establish which. The appropriate behavior is review and diagnosis. This finding illustrates why evidence schemas should preserve component status and why consumers must not collapse a report to one favorable label.

9.3 Event totals and boundaries

The 27 complete traces contain 7,155 counted ledger events, exactly 265 per complete session. Thus the corpus has low structural variation. Each complete trace includes 142 `PolicyEvaluated` events, 44 `PolicyViolation` events, one `ExternalProviderExecutionBlocked` event, one `RuntimeNetworkDenied` event, and one `RuntimeSecretsDenied` event. It also records declaration events for two agents, six policies, two passports, governance, threat, public-conformance, release, ATrust, MCP, and A2A metadata.

The repeated structure supports reproducibility of the pipeline but weakens claims about behavioral diversity. It does not show how the system responds to varied prompts, providers, jurisdictions, policy sets, or attacks. Nor does a recorded denial independently prove host-level enforcement.

9.4 Offline bundle verification

All 27 complete bundles pass the repository’s offline verifier (27/27). Their bytecode, trace, security-report, and trace-event-ledger digest fields are syntactically valid SHA-256 identifiers. The successful result answers RQ3 narrowly: given each bundle and its relative artifact paths, the verifier can reproduce the semantic digests, recompute the ledger digest from trace events, check the `SecurityReportLedgerSummary`’s stored digest against it, and validate cross-artifact linkage without network access. The verifier does not digest or verify `session.argx`; source integrity is outside this result.

It does not answer whether policy approved the request; the detailed policy result is negative. It does not authenticate the producer, prove absence of side effects, or establish semantic correctness of every event. Integrity and approval are orthogonal axes.

Table 7: Principal threats and bounded mitigation claims.

Threat	Mitigation	Claim status
Unauthorized provider execution	Executable-provider allowlist	Implemented
Network side effects	Offline and network-denied pro-files	Implemented
Secret disclosure	Secret and key-material denial	Implemented
Evidence tampering	Digest-linked artifact bundle	Implemented

9.5 Passports and prompts

Every complete session reports two passports, Chile as country and jurisdiction, and Chilean and European Union residency declarations. These values show that sovereign metadata survives to runtime evidence. They do not show that data was physically stored in either region.

The bounded `injected.content` field is present in 0 of the 27 complete, valid traces. The six source-only directories have no trace and are not assessable. This presence metric is independent of the publication allowlist: `prompt_text` remains null unless explicitly allowlisted even when content is present. In this corpus no qualitative prompt themes, quotations, injection labels, or attack success rates can be reported. This is simultaneously a data-minimization benefit and an evaluation limitation.

10 Security Analysis

We analyze threats as attack paths across trust boundaries, not as proof that a named control defeats every adversary. The relevant assets are authorization state, source and bytecode, policy results, provider selection, credentials and secret material, trace/report integrity, and jurisdictional metadata. Adversaries may supply malformed programs, attempt unauthorized operations, tamper with stored artifacts, exploit a provider adapter, or induce a consumer to accept a favorable summary while ignoring detailed failures.

Figure 12 visualizes which boundaries a threat crosses, and table 7 states the narrower mitigation claim supported at each boundary; neither reports measured adversarial efficacy.

Unauthorized provider execution. The split between declarative and executable registries prevents a provider contract from becoming callable merely by being named. Capability checks, approval requirements, and a simulated-only allowlist reduce the observed attack surface. A malicious executable adapter or VM defect remains in scope for future testing.

Network and secret side effects. The demonstrated runtime profile denies network access, external execution, plaintext secrets, and key material. Events make these decisions visible. Because this study does not instrument the host operating system, the evidence supports VM-level denial behavior, not production-grade isolation.

Artifact tampering. Semantic digests detect changes relative to an EvidenceBundle and avoid formatting-dependent hashes. An attacker able to replace both artifacts and the unsigned bundle can create a self-consistent replacement. Signatures, trusted timestamps, independent publication, and key governance are outside the implementation evaluated here.

Policy confusion. The observed status contradiction is an integrity-of-interpretation threat. A dashboard reading only `security_checks` could present an unsafe conclusion even when the detailed object

requires review. Consumers should apply a precedence rule: any explicit failure, unknown rule, or required review prevents an aggregate pass. Future schemas should compute one canonical tri-state decision from typed component results.

Prompt injection and content attacks. Prompt injection is a recognized application risk [9]. ArgorixLangcan constrain resulting operations through capabilities and policy, but this corpus contains no prompt text and no controlled attacks. We make no claim of universal injection prevention and report no success rate.

Identity and trust spoofing. Local semantic checks prevent dangling identity and credential references. They do not establish control of a DID, truth of a credential, or execution of a cryptographic handshake. A future resolver and verifier would introduce new trust anchors, replay concerns, revocation, privacy, and network-failure paths [17, 18].

The strongest supported security statement is therefore conditional: within the observed runtime path, requests are mediated by compiled policy and an executable-provider allowlist, denials are recorded, and complete artifacts are offline-consistent. Security outside that path remains unmeasured.

11 Discussion

RQ1: governance as compilation. The construct inventory shows that identity, residency, provider, policy, governance, threat, and evidence relationships can be represented in one language and rejected before execution when internally inconsistent. The benefit is not that declarations become true; it is that assumptions become typed, referentially checked, and available to both runtime and evidence.

RQ2: constrained request planning. The compiler/VM pipeline and provider split place validation, policy, capability, and registry checks before planning. The observed traces contain blocked external attempts and simulated-only execution. The result supports a fail-closed application architecture, while leaving host isolation untested.

RQ3: evidence and offline consistency. Trace, SecurityReport, and EvidenceBundle serve different functions: event history, interpreted runtime state, and content linkage among bytecode, trace, security report, and the ledger digest derived from trace events. The report’s LedgerSummary stores that digest, not the events. All complete bundles verify offline; the upstream source is not part of that verification. The policy contradiction proves why these artifacts should not be collapsed: valid evidence can faithfully preserve an unfavorable result.

RQ4: demonstrated and undemonstrated behavior. The snapshot demonstrates repeatable artifact production for 27 complete sessions, six visible incomplete sessions, stable control events, and offline digest verification. It does not demonstrate prompt robustness, diverse workloads, live providers, identity authentication, credential verification, cryptographic handshakes, production containment, or compliance.

RQ5: open agentic-web placement. Project NANDA provides a research direction for indexed, verifiable agent facts [10, 11]; DCP-AI frames digital citizenship and governance [7]; ATrust supplies an unpublished conceptual trust vocabulary [15]. ArgorixLangcan compile local metadata that might feed such systems, but it is not their resolver, registry, or trust authority.

This positioning differs from protocol conformance. MCP and A2A define interaction protocols [1, 6]; ArgorixLangpresently checks declarative bridge boundaries with execution disabled. Similarly, DID and

VC standards specify interoperable representations [17, 18]; the language stores descriptions without resolving or verifying them.

The policy contradiction is also a design lesson for evidence-consuming systems. “Passed” should not be a free string replicated at multiple levels. A typed lattice such as *deny* > *review* > *allow*, with unknown treated as review or deny, would make aggregation monotone. An EvidenceBundle should then bind both component results and the derived decision, enabling a verifier to recompute the aggregate.

Finally, the repeated 265-event trace suggests that exhaustive metadata can become noisy. Evidence maps and report summaries can improve navigation, but summary must remain traceable to component events. Compact evidence is useful only if it cannot erase negative findings.

12 Limitations and Claim Boundaries

The dataset is small, single-application, and structurally homogeneous. Its 33 directories are not an independent random sample, and the 27 complete traces have identical event totals. Percentages are descriptive only. Six source-only directories have unknown outcomes, introducing survivor bias if analysis is restricted to complete sessions.

No controlled baseline, adversarial campaign, fault injection, concurrency study, performance benchmark, or usability study was conducted. The bounded prompt-content field was present in 0/27 complete valid traces; six source-only directories were unassessable. Thus prompt-level qualitative analysis and injection measurement are impossible. We do not infer attack success from policy violations: those violations report unknown rules, not successful attacks.

Offline verification checks bytecode, trace, security-report, and trace-event-ledger digest consistency against an unsigned local bundle. It does not check source integrity, authenticate authorship, establish trusted time, prevent replacement of the entire bundle, or prove that the trace is complete. SHA-256 is used for semantic digests; no post-quantum security property is evaluated or claimed.

Passports, `ans_name`, jurisdictions, residency, ATrust identities, credential contracts, handshakes, ledgers, MCP bridges, and A2A bridges are partly or wholly declarative. The current system does not resolve Sovereign DNS/ANS names or DIDs, authenticate real agents, verify credentials, execute cryptographic handshakes, provide blockchain consensus or immutable storage, or run live MCP/A2A interoperability.

Application-level deny flags and the provider registry do not establish production isolation. We did not audit dependencies, prove memory safety of all unsafe/transitive code, inspect operating-system calls, or test container escapes. We make no guarantee against all prompt injection, data leakage, or malicious providers.

Governance mappings are not compliance certification. Neither the language nor this study certifies conformity with a law, regulator, NIST AI RMF, or any other framework [13]. The empirical contradiction between the top-level and detailed statuses further prevents a positive assurance conclusion.

Future evaluations should publish controlled workloads, independent verifiers, OS-level telemetry, and pre-registered outcome definitions.

13 Future Work: Sovereign Discovery

The proposed Sovereign DNS/Agent Naming Service would resolve an `ans_name` to a versioned agent record containing endpoints, passport references, supported protocols, jurisdictional metadata, and evidence locations. It would draw on DNS lessons concerning hierarchical delegation [4, 5], NANDA’s agent-index direction [11], and DID/VC representations [17, 18]. This is proposed architecture, not implemented resolution.

A credible implementation requires more than a lookup API. It needs namespace governance, proof of control, authenticated record updates, revocation, key rotation, replay protection, privacy-preserving

queries, caching semantics, conflict handling, auditability, and resilience to compromised registries. Jurisdiction and residency assertions need issuer and evidence policies rather than self-assertion alone.

ATrust contracts could define which identity methods, credential types, and handshake evidence a caller requires. DCP-AI could inform citizenship and control-plane semantics. ArgorixLang could compile these requirements and reject a discovered record before an adapter is planned. An executable resolver, credential verifier, and cryptographic handshake engine would nevertheless be new trusted components, requiring independent threat models and tests.

Empirically, future work should generate a controlled matrix: valid and invalid records, stale keys, revoked credentials, cross-jurisdiction residency conflicts, unavailable registries, malicious adapters, altered artifacts, and summary/detail contradictions. Outcome measures should include denial correctness, false acceptance, false rejection, verification latency, evidence size, and recovery behavior. Prompt robustness should be studied only with an explicitly collected, privacy-reviewed corpus.

For evidence, signed bundles and append-only transparency publication could provide producer authentication and replacement detection, subject to key governance and availability analysis. This direction should not be described as blockchain immutability unless an actual consensus/storage system is implemented and evaluated.

14 Conclusion

ArgorixLang demonstrates how agent declarations, policy, sovereign metadata, provider boundaries, and evidence requirements can share a compiled systems boundary. In an authorized observational snapshot, 27 of 33 request directories were complete, six were source-only, and all 27 complete EvidenceBundles verified bytecode, trace, security-report, and trace-event-ledger digest consistency offline; source was outside verification. The same complete sessions contained 1,188 detailed policy violations and uniformly required review despite a favorable top-level status. That contradiction is not incidental: it establishes the paper’s central distinction between integrity and approval.

The implemented contribution is a validated language, fail-closed VM path, simulated-only executable-provider boundary, runtime trace, SecurityReport, EvidenceBundle, and semantic digest verification. Passports and `ans_name` are implemented as declarative identifiers and metadata. ATrust, MCP, and A2A relationships are validated declarations, not live trust or protocol execution. Sovereign DNS/ANS resolution is proposed. Real identity authentication, credential verification, cryptographic handshakes, blockchain immutability, post-quantum security, compliance certification, universal injection prevention, and production isolation are not claimed.

The broader systems lesson is that trustworthy evidence must preserve negative and incomplete outcomes. A verifiable bundle is valuable precisely because it can carry a failed policy decision without relabeling it as success.

References

- [1] A2A Protocol Working Group. Agent2agent (A2A) protocol specification. Protocol specification, 2026. URL <https://a2a-protocol.org/v1.0.0/specification/>.
- [2] Argorix Labs. ArgorixLang, 2026. URL <https://github.com/argorixlabs/argorixlang>. Secure, verifiable programming language and runtime for AI-agent systems.
- [3] Rafael H. Bordini, Jomi Fred Hübner, and Michael Wooldridge. *Programming Multi-Agent Systems in AgentSpeak using Jason*. John Wiley & Sons, 2007. ISBN 9780470061848. doi: 10.1002/9780470061848. URL <https://doi.org/10.1002/9780470061848>.
- [4] Paul Mockapetris. Domain names—concepts and facilities. RFC 1034, Internet Engineering Task Force, November 1987. URL <https://www.rfc-editor.org/rfc/rfc1034.html>.

- [5] Paul Mockapetris. Domain names—implementation and specification. RFC 1035, Internet Engineering Task Force, November 1987. URL <https://www.rfc-editor.org/rfc/rfc1035.html>.
- [6] Model Context Protocol Contributors. Model context protocol specification, version 2025-06-18. Protocol specification, 2025. URL <https://modelcontextprotocol.io/specification/2025-06-18>.
- [7] Danilo Naranjo Emparanza. Agents don’t need a better brain—they need a world: Toward a digital citizenship protocol for autonomous AI systems. Engineering Archive preprint, 2026. URL <https://engrxiv.org/preprint/view/6671>. DCP-AI: Digital Citizenship Protocol for Autonomous AI.
- [8] Open Policy Agent Project. Open policy agent (OPA). Official project documentation, 2026. URL <https://www.openpolicyagent.org/docs>. General-purpose policy engine and Rego policy-as-code language.
- [9] OWASP Foundation. OWASP top 10 for large language model applications. Security guidance, 2025. URL <https://owasp.org/www-project-top-10-for-large-language-model-applications/>.
- [10] Project NANDA. Architecting the internet of AI agents. Project website, 2026. URL <https://projectnanda.org/>. Project NANDA (Networked AI Agents in Decentralized Architecture).
- [11] Ramesh Raskar, Pradyumna Chari, John Zinky, Mahesh Lambe, Jared James Grogan, Sichao Wang, Rajesh Ranjan, Rekha Singhal, Shailja Gupta, Robert Lincourt, Raghu Bala, Aditi Joshi, Abhishek Singh, Ayush Chopra, Dimitris Stripelis, Bhuwan B, Sumit Kumar, and Maria Gorskikh. Beyond DNS: Unlocking the internet of AI agents via the NANDA index and verified AgentFacts. *arXiv*, 2025. doi: 10.48550/arXiv.2507.14263. URL <https://arxiv.org/abs/2507.14263>.
- [12] Rust Project. Rust programming language. Official project website, 2026. URL <https://rust-lang.org/>.
- [13] Elham Tabassi. Artificial intelligence risk management framework (AI RMF 1.0). NIST AI 100-1, National Institute of Standards and Technology, January 2023. URL <https://doi.org/10.6028/NIST.AI.100-1>.
- [14] Santiago Torres-Arias, Hammad Afzali, Trishank Karthik Kuppusamy, Reza Curtmola, and Justin Cappos. in-toto: Providing farm-to-table guarantees for bits and bytes. In *28th USENIX Security Symposium (USENIX Security 19)*, pages 1393–1410, Santa Clara, CA, August 2019. USENIX Association. ISBN 978-1-939133-06-9. URL <https://www.usenix.org/conference/usenixsecurity19/presentation/torres-arias>.
- [15] Edison Vazquez. ATrust: Agentic trust protocol. Author-supplied metadata used in this study; unpublished conceptual protocol.
- [16] WASI Subgroup. Introduction. Official standards portal, 2026. URL <https://wasi.dev/>. Standards-track APIs and capability-based sandbox model.
- [17] World Wide Web Consortium. Decentralized identifiers (DIDs) v1.0: Core architecture, data model, and representations. W3C recommendation, World Wide Web Consortium, July 2022. URL <https://www.w3.org/TR/2022/REC-did-core-20220719/>.
- [18] World Wide Web Consortium. Verifiable credentials data model v2.0. W3C recommendation, World Wide Web Consortium, May 2025. URL <https://www.w3.org/TR/2025/REC-vc-data-model-2.0-20250515/>.

A Bounded Artifact Excerpts

The following excerpts are schematic projections of fields present in the normalized data. Values that identify individual requests are omitted; no prompt, secret, endpoint, absolute path, full log, or identifier dump is included.

Listing 1: Redacted session-level projection.

```
{
  "complete": true,
  "artifact_count": 5,
  "execution_status": "completed",
  "security_checks": "passed",
  "policy_passed": false,
  "review_required": true,
  "policy_violation_count": 44,
  "prompt_content_present": false,
  "prompt_text": null
}
```

Listing 2: Redacted provider-boundary projection.

```
{
  "declarative_contract_names": ["OpenAIProvider"],
  "executable_providers": ["simulated"],
  "external_contracts_total": 1,
  "blocked_attempts": 1,
  "external_execution_blocked": true
}
```

Listing 3: Bounded verification-result projection.

```
{
  "exit_code": 0,
  "verified": true,
  "stdout": "Evidence verification: passed",
  "stderr": ""
}
```

B Reproduction Commands

From the root of this paper worktree, the authorized artifact snapshot resides in the primary checkout two levels above. The following commands use only root-relative paths and provide every mandatory analyzer argument:

```
python -m pip install -r paper/requirements.txt
powershell -NoProfile -ExecutionPolicy Bypass -File paper/scripts/build_paper.ps1 -Target all -InputRoot
  ../../demo/argorix-chatbot-runtime/generated
```

In a conventional checkout where generated artifacts are directly under the repository root, replace each `../../demo/` prefix with `demo/`. The runtime analyzer fails visibly if the input is absent. The explicit `-InputRoot` prevents a worktree's tracked but empty `generated/.gitkeep` directory from being mistaken for the authorized snapshot. To rebuild only the PDF with the pinned, checksum-verified Tectonic engine, use:

```
powershell -NoProfile -ExecutionPolicy Bypass -File paper/scripts/build_paper.ps1 -Target paper
```

The committed normalized data permit manuscript and figure regeneration without publishing the original request-directory names or prompt content.

Fail-closed decision state machine

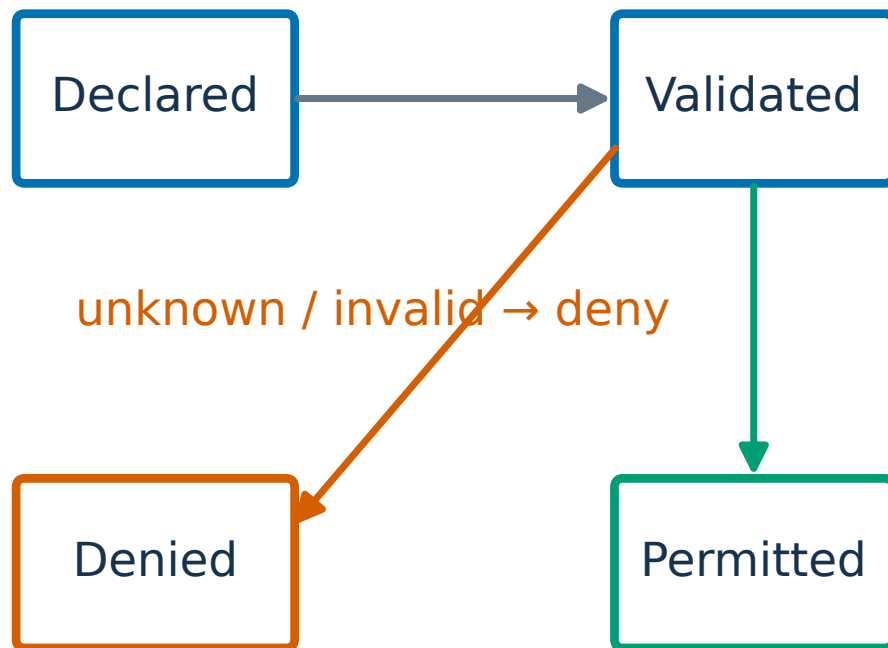


Figure 7: Fail-closed decision state machine. Only validated, policy-permitted operations with an executable provider reach execution; every other branch terminates with an auditable denial or review state.

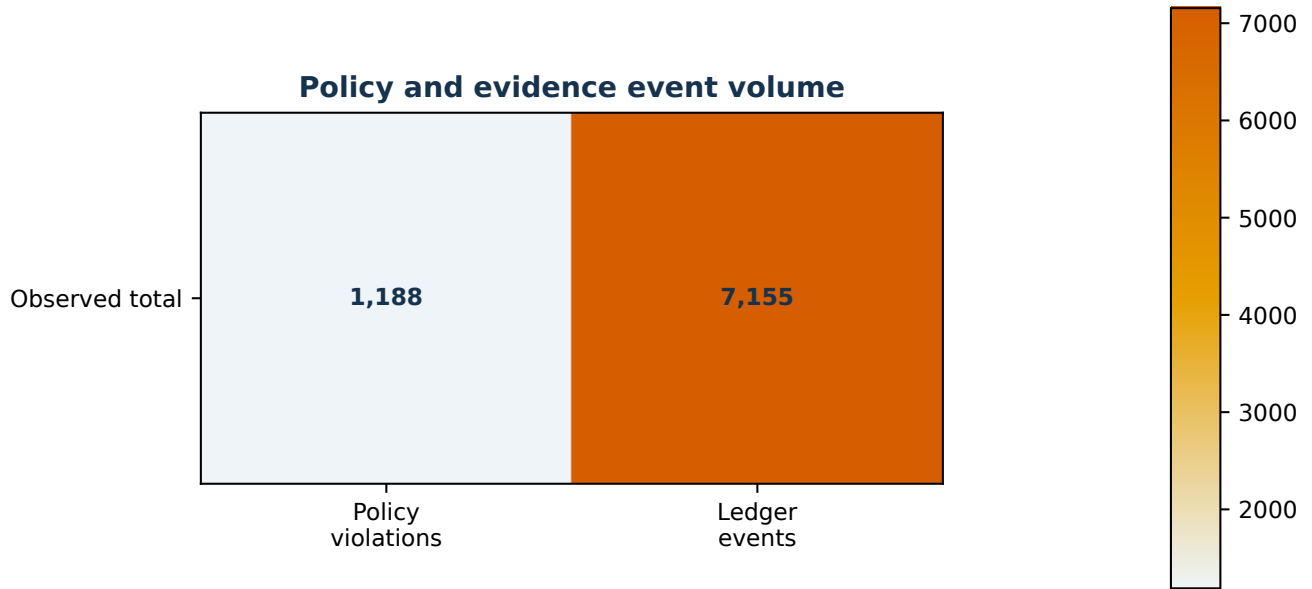


Figure 8: Observed control and policy states across normalized sessions. Blank or unavailable cells for source-only directories are missing observations, not passes.

Normalized artifact relationships



Figure 9: Schema-level view of the five-artifact request directory. Source is an upstream input outside EvidenceBundle verification; the bundle references bytecode, trace, and security-report semantic digests plus a ledger digest derived from trace events, and does not contain a security verdict.

Claim boundary taxonomy



Figure 10: Claim-boundary plate separating implemented mechanisms, declarative metadata, proposed architecture, and properties not claimed.

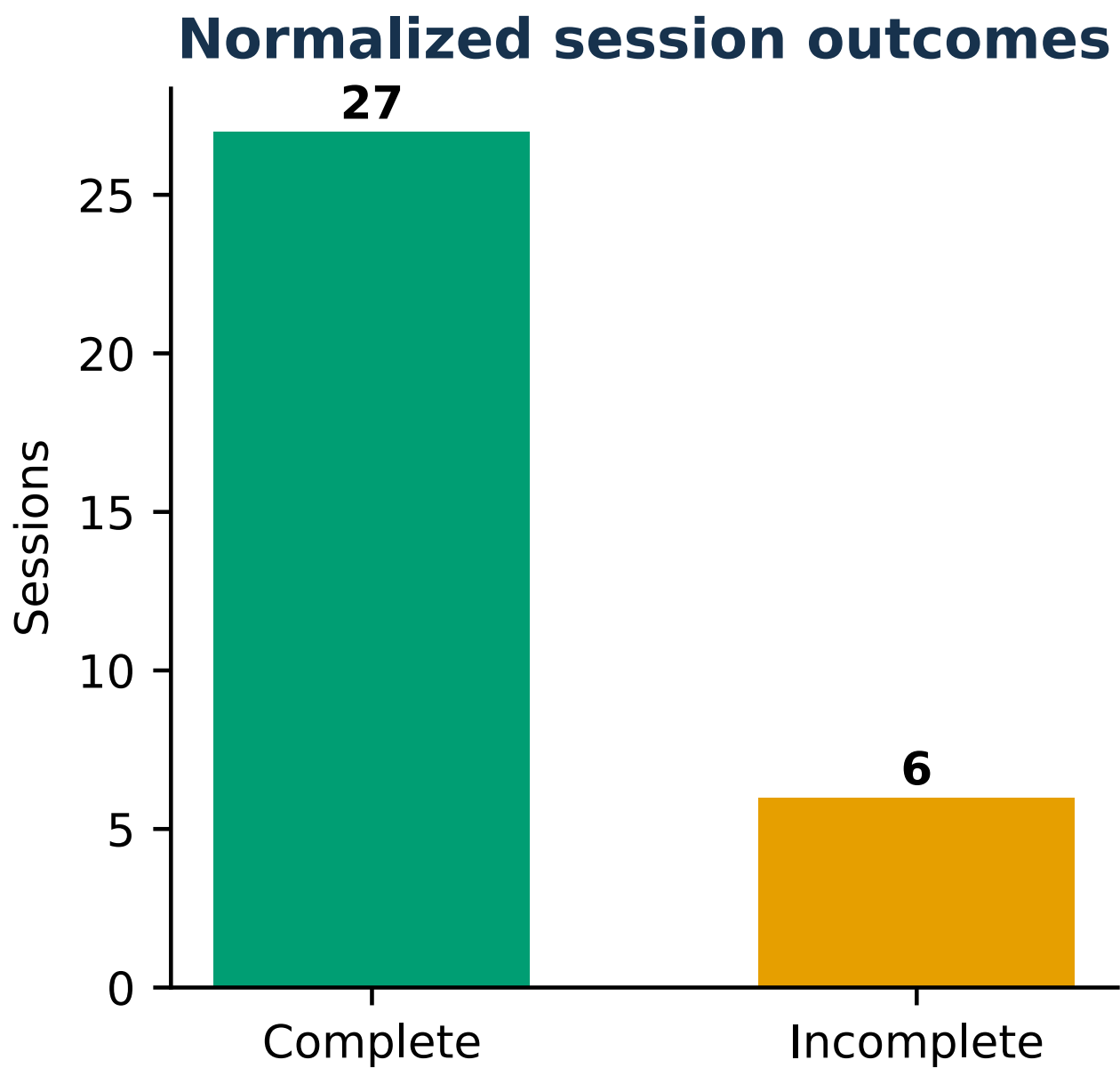


Figure 11: Artifact completeness and observed outcomes for all 33 request directories. Source-only observations remain explicit negative/incomplete results.

Threat-to-control mapping

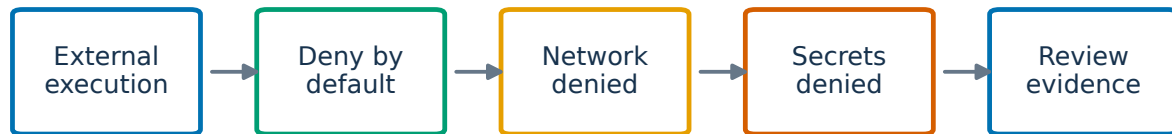


Figure 12: Threat-mitigation mapping for the evaluated architecture. Mapping means a control is present or declared; it is not a measured attack-blocking rate.